

Experiment7

- 1) Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Code :

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 10; // Global variable

void code_function()
{   printf("Code address  : %p\n", (void
*)code_function);
}

int main() {   static int static_var = 20; //
Static variable   int stack_var = 30;      // Stack
variable   int *heap_var = malloc(sizeof(int)); //
Heap variable

    if (heap_var == NULL)
    {
        printf("Memory allocation failed\n");
return 1;
    }

    *heap_var = 40;

    printf("Linux Process Address Space\n");   printf("----
-----\n");

    code_function();

    printf("Global address : %p\n", (void *)&global_var);
printf("Static address : %p\n", (void *)&static_var);
printf("Heap address  : %p\n", (void *)heap_var);   printf("Stack
address : %p\n", (void *)&stack_var);   free(heap_var);
```

```
    return 0;
}
```

OUTPUT :

```
venkat038@AYYAPPA:~$ nano 7.1.c
venkat038@AYYAPPA:~$ gcc 7.1.c -o 7.1
venkat038@AYYAPPA:~$ ./7.1
Linux Process Address Space
-----
Code address : 0x635bfc0651c9
Global address : 0x635bfc068010
Static address : 0x635bfc068014
Heap address : 0x635c29ce2010
Stack address : 0x7ffc6bb4259c
venkat038@AYYAPPA:~$ |
```

2) Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

int global_var = 100;

int main()

{
```

```

static int static_var = 200;  int
stack_var = 300;  int *heap_var =
malloc(sizeof(int));

*heap_var = 400;    printf("Process ID (PID): %d\n",
getpid());  printf("Global variable address : %p\n", (void
*)&global_var);  printf("Static variable address : %p\n", (void
*)&static_var);  printf("Heap variable address : %p\n", (void
*)&heap_var);  printf("Stack variable address : %p\n", (void
*)&stack_var); printf("\nProcess is running...\n");  getchar();
free(heap_var); return 0;
}

```

OUTPUT:

```

venkat038@AYYAPPA:~$ nano 7.2.c
venkat038@AYYAPPA:~$ gcc 7.2.c -o 7.2
venkat038@AYYAPPA:~$ ./7.2
Process ID (PID): 872
Global variable address : 0x57ad8a87f010
Static variable address : 0x57ad8a87f014
Heap variable address : 0x57ad9cf47010
Stack variable address : 0x7ffcd929620c

Process is running...

```